

UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

DIPARTIMENTO DI INGEGNERIA “*ENZO FERRARI*”

Laurea Triennale in Ingegneria Informatica

Implementazione di un sistema multimodale basato su Catene di Markov

Relatore:

Francesco Guerra

Candidato:

Francesco Lasalvia

Anno Accademico 2024/2025

Indice

Introduzione	3
1 Descrizione degli elementi	5
1.1 Catene di Markov	5
1.1.1 Definizione	5
1.1.2 Proprietà	6
2 Applicazioni delle catene di Markov	8
2.1 Primo caso d'uso: Testo	8
2.1.1 Implementazione	8
2.1.2 Risultati	13
2.1.3 Diagrammi UML	14
2.2 Secondo caso d'uso: Dati numerici	15
2.2.1 Implementazione	15
2.2.2 Risultati	17
2.2.3 Diagrammi UML	18
2.3 Terzo caso d'uso: Paradigma Client-Server	20
2.3.1 Implementazione	20
2.3.2 Risultati	27
2.3.3 Diagrammi UML	32
3 Considerazioni Finali	35
4 Ringraziamenti	37

Elenco delle figure

1.1	Struttura generale Catena di Markov.	5
2.1	Activity Diagram.	14
2.2	Class Diagram.	18
2.3	Sequence Diagram.	19
2.4	Istogrammi delle prestazioni del server.	32

Introduzione

La stesura del documento procederà con un'analisi del concetto di catena di Markov, con relative proprietà. Tali concetti verranno applicati per le seguenti casistiche:

- un testo,
- alcuni dati numerici,
- il paradigma client-server.

Per ciascun caso saranno illustrate, mediante diagrammi UML, le ragioni alla base della struttura del codice. Verrà quindi presentato il concetto di catena di Markov, ne saranno descritte le proprietà e le caratteristiche matematiche, e saranno messe a confronto le relative applicazioni con le pratiche dell'ingegneria del software. L'analisi progettuale proseguirà con ulteriori diagrammi UML e con l'esposizione di alcuni design pattern essenziali per garantire robustezza e scalabilità alle soluzioni implementate. Infine, si forniranno le conclusioni e si esprimeranno i ringraziamenti.

Capitolo 1

Descrizione degli elementi

1.1 Catene di Markov

1.1.1 Definizione

Le catene di Markov sono modelli matematici utilizzati per descrivere sistemi che evolvono nel tempo seguendo un processo stocastico [1]. La caratteristica principale di questi modelli è che lo stato futuro del sistema dipende esclusivamente dallo stato attuale, senza considerare gli eventi passati, proprietà nota come “**memoria limitata**” o “**proprietà markoviana**” [2]. In sostanza, il processo viene descritto come una serie di stati in cui le transizioni tra stati avvengono secondo determinate probabilità [3].

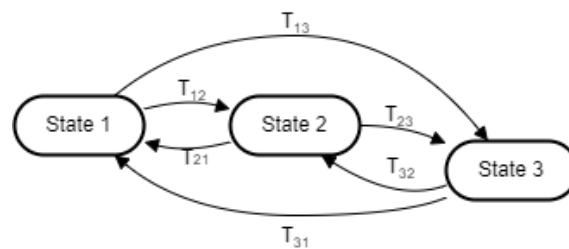


Figura 1.1: Struttura generale Catena di Markov.

Definizione matematica

Matematicamente, un processo stocastico X_t è detto di Markov se, per ogni $t \leq u$ e per ogni x reale, abbiamo che:

$$P(X_u \leq x \mid X_s, s \leq t) = P(X_u \leq x \mid X_t)[3].$$

Questa è la **proprietà di Markov**, la quale implica che la probabilità che il processo si trovi in un certo stato al tempo $t + 1$ dipende solo dallo stato corrente al tempo t , indipendentemente dagli stati precedenti:

$$P(X_{t+1} = s_{t+1} \mid X_t = s_t, X_{t-1} = s_{t-1}, \dots, X_0 = s_0) = P(X_{t+1} = s_{t+1} \mid X_t = s_t)[4].$$

Questa proprietà è nota anche come *memoria corta* della catena di Markov.

1.1.2 Proprietà

Irreducibilità

Una catena di Markov è **irriducibile** se è possibile raggiungere qualsiasi stato da ogni altro stato in un numero finito di passi. In altre parole, tutti gli stati comunicano tra di loro [4].

Ricorrenza e Transitorietà

Uno stato è **ricorrente** se esiste una probabilità non nulla di ritornarvi partendo da esso. Se invece la probabilità di ritornarvi è nulla, lo stato è **transitorio**. Se uno stato ricorrente viene visitato infinitamente spesso, esso è detto **positivamente ricorrente**. Se il tempo medio di ritorno è infinito, lo stato è **null ricorrente** [4].

Periodicità

Uno stato si dice **periodico** se la catena ritorna in quello stato solo in tempi che sono multipli di un numero fisso $d \geq 2$. Se $d = 1$, lo stato è **aperiodico**. Se tutti gli stati sono aperiodici, allora l'intera catena è aperiodica [3].

Ergodicità

Una catena di Markov è **ergodica** se è irriducibile, aperiodica e tutti gli stati sono positivamente ricorrenti. In questo caso, esiste una distribuzione stazionaria unica verso cui la catena converge indipendentemente dallo stato iniziale [3].

Distribuzione Stazionaria

La **distribuzione stazionaria** è una distribuzione probabilistica π sugli stati della catena tale che:

$$\pi P = \pi$$

Una volta che la catena raggiunge la distribuzione stazionaria, essa rimane in quella distribuzione per tutti i tempi successivi.[4]

Reversibilità

Una catena di Markov è **reversibile** se la distribuzione stazionaria π e la matrice di transizione P soddisfano la condizione di reversibilità:

$$\pi(s_i)P(s_i, s_j) = \pi(s_j)P(s_j, s_i)$$

Questa proprietà è utile per dimostrare l'esistenza di una distribuzione stazionaria.[3]

Capitolo 2

Applicazioni delle catene di Markov

2.1 Primo caso d'uso: Testo

2.1.1 Implementazione

Panoramica generale

In questa implementazione, utilizzeremo il concetto di catena di Markov applicato ad un testo. Le parole sono state definite come sequenze di caratteri racchiuse tra due spazi, ovvero il carattere ' ', la punteggiatura verrà ignorata. Gli stati sono rappresentati da sequenze di parole, le quali verranno memorizzate all'interno di un dizionario, creato attraverso una mappatura di una lista di stringhe. Ogni stato è rappresentato da una chiave nel dizionario Markov, che è una lista di parole di dimensione 'keySize'. Introduciamo quindi il concetto di transizioni: esse sono le probabilità che una determinata sequenza di parole (**stato**) sia seguita da una parola successiva.

Richiesta dei parametri

Viene richiesto all'utente la dimensione della keysize e la dimensione del testo in output da generare.

Definizione probabilità

Una volta definito il testo di riferimento, le probabilità sono stimate osservando la frequenza con cui ogni parola segue una determinata sequenza di parole. In particolare, si noti che la probabilità di una parola in una sequenza dipenda solo dalle 'keySize' parole precedenti, e non dalla sequenza di parole più lontane. Questo è proprio il principio Markoviano. Andiamo a vedere ora in dettaglio il codice.

Metodo markov

Dopo la richiesta dei parametri, si procede al richiamo di quello che rappresenta la vera essenza del codice.

```

1  public static String markov(int keySize, int outputSize) throws IOException {
2      //Generazione parole
3      List<String> words = Arrays.asList(new String(Files.readAllBytes(Paths.get(
        searchFolder("book")+"\\ "+fileNoExtension+".txt"))).trim().split("\\s+"));
4
5      //Costruzione del dizionario
6      Map<List<String>, List<String>> dict = buildMarkovDict(keySize, words);
7
8      //Genera output iniziale
9      List<String> output = generateInitialOutput(words, keySize);
10
11     //Genera il testo output
12     StringBuilder result = generateText(dict, output, outputSize, keySize);
13
14     //Aggiusta il testo
15     handleEndOfSentence(result);
16
17     //Stampa info
18     printAdditionalInfo(outputSize);
19
20     //Ritorna l'output
21     return result.toString().trim();
22 }

```

Codice 2.1: Costruzione del dizionario di Markov.

Possiamo identificare cinque sezioni di codice:

- la prima sezione di codice legge tutto il contenuto di un file di testo, lo trasforma in una stringa, rimuove eventuali spazi bianchi ai bordi, divide il testo in parole basandosi sugli spazi e memorizza le parole in una lista (*List<String>*) chiamata *words*,
- la seconda sezione di codice realizza il dizionario di parole:

```

1  private static Map<List<String>, List<String>> buildMarkovDict(int keySize
2      , List<String> words) {
3      /*
4      Inizializzazione del dizionario:
5      Crea un LinkedHashMap chiamato dict che mantiene l'ordine d'
        inserimento delle chiavi.
6      */
7      Map<List<String>, List<String>> dict = new LinkedHashMap<>();
8
9      for (int i = 0; i < words.size() - keySize; i++) {
10         /*
            Costruzione delle chiavi:

```

```
11         Per ogni iterazione del ciclo, crea una chiave List<String>
chiamata key utilizzando il metodo subList sulla lista delle parole words.
12         Questo estrae una sottolista di parole di lunghezza keySize a
partire dall'indice i.
13         */
14         List<String> key = words.subList(i, i + keySize);
15         /*
16         Aggiunta dei valori al dizionario: La variabile value viene
inizializzata con la parola successiva alla sequenza corrente di parole (
se disponibile).
17         Quindi, utilizzando computeIfAbsent, aggiunge value alla lista dei
valori associati alla chiave key nel dizionario.
18         Se la chiave non e' presente, crea una nuova lista di valori per
quella chiave e aggiunge il valore a essa.
19         */
20         String value = (i + keySize < words.size()) ? words.get(i +
keySize) : "";
21         dict.computeIfAbsent(key, k -> new ArrayList<>()).add(value);
22     }
23     return dict;
24 }
```

Codice 2.2: Costruzione del dizionario di Markov.

- la terza sezione di codice crea la stringa di partenza del testo output,
- la quarta sezione di codice crea il resto del testo:

```
1     private static StringBuilder generateText(Map<List<String>, List<String>>
dict, List<String> output, int outputSize, int keySize) {
2         // Usa StringBuilder per costruire il testo risultante
3         StringBuilder result = new StringBuilder();
4
5         // Variabile per determinare se la prossima parola deve iniziare con
una lettera maiuscola
6         boolean capitalizeNext = true;
7
8         // Ciclo attraverso le parole gia' presenti in output
9         for (String word : output) {
10             // Capitalizza la parola se necessario
11             word = capitalizeFirstLetter(word, capitalizeNext);
12             // Aggiungi la parola al risultato
13             result.append(word).append(" ");
14
15             // Se la parola termina con un punto, aggiungi una nuova riga e
capitalizza la prossima parola
16             if (word.endsWith(".")) {
17                 result.append("\n");
18                 capitalizeNext = true;
19             } else {
20                 capitalizeNext = false;
21             }
22             // Incrementa il contatore delle parole generate se la parola non
e' una punteggiatura
```

```
23         if (!word.matches("\\p{Punct}")) {
24             wordsGenerated++;
25         }
26     }
27     // Continua a generare parole fino a raggiungere la dimensione
    desiderata dell'output
28     while (wordsGenerated < outputSize) {
29         // Ottieni la lista di parole successive basata sulle ultime '
    keySize' parole in output
30         List<String> suffix = dict.get(output.subList(output.size() -
    keySize, output.size()));
31
32         // Se non ci sono parole successive disponibili, interrompi il
    ciclo
33         if (suffix == null || suffix.isEmpty()) {
34             break;
35         }
36
37         // Scegli la prossima parola basata su una scelta ponderata
38         String nextWord = weightedRandomChoice(suffix);
39         // Capitalizza la prossima parola se necessario
40         nextWord = capitalizeFirstLetter(nextWord, capitalizeNext);
41         // Aggiungi la parola alla lista di output
42         output.add(nextWord);
43
44         // Controlla se la parola e' una punteggiatura o una rottura di
    linea
45         boolean isPunctuation = nextWord.matches("\\p{Punct}");
46         boolean isLineBreak = nextWord.equals("\n");
47
48         // Se la parola non e' punteggiatura
49         if (!isPunctuation) {
50             if (isLineBreak) {
51                 // Se e' una rottura di linea, aggiungi una nuova riga al
    risultato e capitalizza la prossima parola
52                 result.append("\n");
53                 capitalizeNext = true;
54             } else {
55                 // Altrimenti, aggiungi la parola e un spazio al risultato
56                 result.append(nextWord).append(" ");
57                 // Se la parola termina con un punto, aggiungi una nuova
    riga e capitalizza la prossima parola
58                 if (nextWord.endsWith(".")) {
59                     result.append("\n");
60                     capitalizeNext = true;
61                 } else {
62                     capitalizeNext = false;
63                 }
64                 wordsGenerated++; // Incrementa il contatore delle parole
    generate
65
66                 // Controlla se il numero di parole generate ha raggiunto
    la dimensione desiderata
67                 if (wordsGenerated >= outputSize) {
68                     break;
69                 }
70             }
```

```

71     }
72 }
73     return result; // Ritorna il testo generato
74 }

```

Codice 2.3: Costruzione del dizionario di Markov.

- la quinta sezione è realizzata dalla gestione delle frasi e da stampe aggiuntive.

```

1     private static void handleEndOfSentence(StringBuilder result) {
2         // Controllo se la stringa è vuota
3         if (result == null) {
4             return;
5         }
6
7         // Controllo se l'ultima frase ha un punto
8         int lastDotIndex = result.lastIndexOf(".");
9         if (lastDotIndex != -1 && lastDotIndex == result.length() - 1) {
10             // Se l'ultima frase termina con un punto, non fare nulla
11             return;
12         }
13
14         // Se l'ultima frase non ha un punto, trovare l'ultimo punto
15         int lastPeriodIndex = result.lastIndexOf(".", lastDotIndex - 1);
16
17         // Controllo se è stato trovato un punto in una posizione valida
18         if (lastPeriodIndex != -1) {
19             // Eliminare tutto quello che segue il punto e contare le parole
20             // eliminate
21             String wordsAfterLastPeriod = result.substring(lastPeriodIndex +
22             1, result.length());
23             String[] words = wordsAfterLastPeriod.split("[\\s\\n]+"); //
24             Dividere le parole per spazio o nuova riga
25             for (String word : words) {
26                 // Controllo se la parola contiene solo lettere o numeri (
27                 // ignorando punteggiatura e \n)
28                 if (word.matches("[a-zA-Z0-9]+")) {
29                     wordsCut++;
30                 }
31             }
32             result.delete(lastPeriodIndex + 1, result.length());
33         }
34         // Stampare il numero di parole eliminate
35         if (getWordsCut() > 0)
36             System.out.println("Ho eliminato alcune parole per un finale
37             migliore!\n" +
38                 "Ho tagliato: " + getWordsCut() + " parole!\n");
39         cornice();
40     }

```

Codice 2.4: Costruzione del dizionario di Markov.

2.1.2 Risultati

Di seguito, un esempio di output del codice:

```

1 Premi invio per cominciare la simulazione!
2 +-----+
3 Benvenuti alla mia implementazione di una Markov Chain basata su un testo!
4 +-----+
5 Cominciamo dal leggere i file disponibili!
6
7 1. Il Signore Degli anelli
8 2. Never Gonna Give You Up
9 3. Sherlock Holmes Adventure
10 +-----+
11 Inserisci il numero corrispondente al file desiderato: 3
12 +-----+
13 File scelto: Sherlock Holmes Adventure
14 +-----+
15 Per prima cosa, mi serve la dimensione della chiave, scrivila sul terminale per
    favore.
16 Piu' e' grande, maggiore sara' il senso del testo!
17 Non accetto zero come parametro!
18 Ti consiglio di darmi un numero tra 8 e 15, non piu' di 20!
19 Inserisci la dimensione della chiave: 5
20 +-----+
21 Ora mi serve sapere quante parole vuoi che generi.
22 Ricorda che non posso generare piu' parole di quelle che mi hai dato, che sono
    esattamente: 30874 parole!
23 Inserisci la dimensione dell'output: 100
24 Ora ho la chiave di grandezza: 5 e la grandezza dell'output finale di : 100
25 +-----+
26 Inizio la nuova generazione...attendere prego...
27 ... ..
28 ECCO A VOI IL NUOVO CAPITOLO DI Sherlock Holmes Adventure
29 +-----+
30 Ho eliminato alcune parole per un finale migliore!
31 Ho tagliato: 4 parole!
32 +-----+
33 Inizialmente hai richiesto di stampare: 100 parole.
34 Io ho generato: 96 parole!
35 Ma ricorda che ho tagliato: 4 parole!
36 Infatti 100 - 4 fa: 96!
37 +-----+
38 Possibile a quel tormento.
39 Qualche volta anche, per farla tacere, per impedirle di parlare, per ricacciare
    quasi la domanda, che stava per spuntarle sulle labbra, egli le prendeva la
    testa con le mani, avvicinava la bocca alla sua bocca, la soffocava di baci: e
    quando, stordita e sorridente, si riaveva, non lo trovava piu' vicino a lei.
40 Egli era gia' partito.
41 Ed ella indovinava allora il perche' di quella furia di carezze, di quella foga
    affettuosa, di quella finzione. E quel sorriso, quella gioia, quella felicita'
    erano sempre seguiti da un lungo scoppio di pianto.
42 +-----+
43 Premi invio per terminare la simulazione!

```

Codice 2.5: Output Markov su testo.

2.1.3 Diagrammi UML

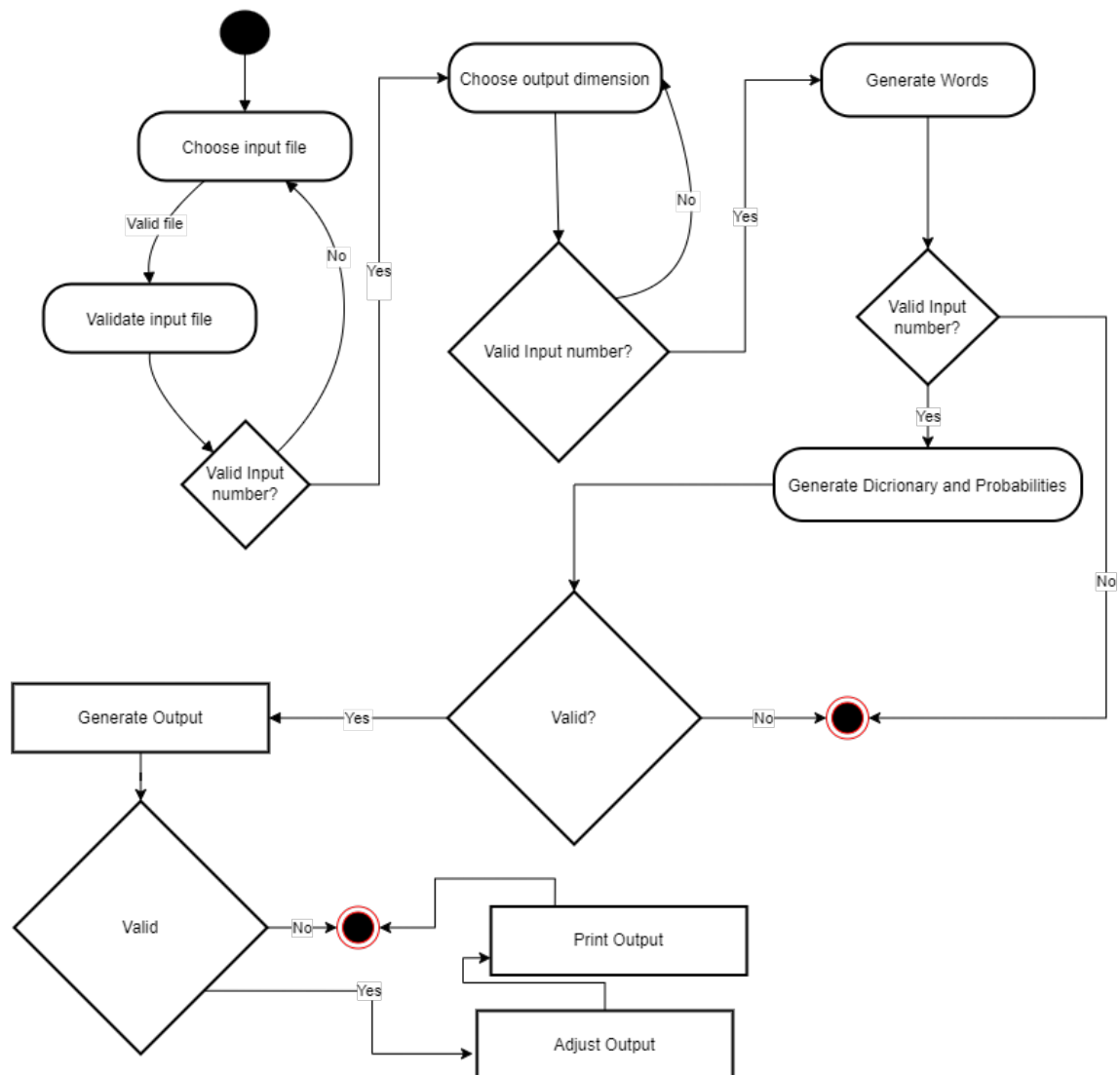


Figura 2.1: Activity Diagram.

2.2 Secondo caso d'uso: Dati numerici

2.2.1 Implementazione

Panoramica generale

In questa implementazione, utilizzeremo il concetto di catena di Markov applicato a dei dati numerici. Si procederà a simulare, sotto un aspetto prettamente matematico, la diffusione del covid su una popolazione fittizia. Gli stati rappresentano le condizioni della popolazione: *'sani'*, *'infetti'*, *'guariti'*, e *'deceduti'*. Le probabilità di transizione quantificano la possibilità di passare da uno stato a un altro in uno specifico intervallo di tempo. Per esempio, una probabilità di transizione del 20% per diventare infetto implica che ogni persona sana ha il 20% di probabilità di diventare infetta nel prossimo intervallo di tempo.

Richiesta dei parametri

Viene richiesto all'utente il numero della popolazione e il numero di giorni per la quale la simulazione verrà effettuata. La simulazione parte dal giorno 0, indicando il numero totale di sani.

Definizione delle probabilità

Definiamo le probabilità per ogni stato, definite in modo statico:

- *'probTransizioneGuarito'*: Probabilità che una persona infetta guarisca - 80%,
- *'probTransizioneDeceduto'*: Probabilità che una persona infetta deceda. - 3%,
- *'probTransizioneInfetto'*: Probabilità che una persona sana diventi infetta - 20%,
- *'probTransizioneReInfezione'*: Probabilità che una persona guarita possa essere reinfettata. - 15%.

Si osserva come la probabilità di un individuo di diventare infetto, guarire, o decedere non dipende dalla sua storia di transizioni passate ma solo dal suo stato attuale. Ci si accorge di come questo corrisponda al principio Markoviano.

Metodo simulazioneMarkov

Dopo la richiesta dei parametri, si procede con quello che è il cuore di questa simulazione.

```
1     private static void simulazioneMarkov(int sani, int giorniTotali, double
    probTransizioneInfetto, double probTransizioneGuarito, double
    probTransizioneDeceduto, double probTransizioneReInfezione) throws
    InterruptedException {
2         int infetti = 0; // Popolazione infetta all'inizio
3         int guariti = 0; // Popolazione guarita all'inizio
4         int deceduti = 0; // Popolazione deceduta all'inizio
5         int popolazioneTotale = sani; // La popolazione totale e' inizialmente
    uguale alla popolazione sana
6
7         // Giorno 0: popolazione interamente sana
8         System.out.println("Il Primo giorno, la popolazione e' totalmente sana.");
9         System.out.println("Giorno 0: Sani = " + sani + ", Infetti = " + infetti +
    ", Guariti = " + guariti + ", Deceduti = " + deceduti);
10        Thread.sleep(3000);
11        cornice(); // Stampa un separatore per migliorare la leggibilita'
12
13        // Loop che simula il progresso della malattia per il numero di giorni
    inserito dall'utente
14        for (int i = 1; i <= giorniTotali; i++) {
15            // Calcolo del numero di nuove persone che transizionano da uno stato
    all'altro
16            double nuoviInfettiSani = sani * probTransizioneInfetto;
17            double nuoviInfettiGuariti = guariti * probTransizioneReInfezione;
18            double nuoviGuariti = infetti * probTransizioneGuarito;
19            double nuoviDeceduti = infetti * probTransizioneDeceduto;
20
21            // Aggiornamento degli stati: diminuzione dei sani, aumento degli
    infetti, guariti e deceduti
22            sani -= (int) nuoviInfettiSani;
23            infetti += (int) nuoviInfettiSani + (int) nuoviInfettiGuariti;
24            infetti -= (int) nuoviGuariti;
25            guariti += (int) nuoviGuariti;
26            deceduti += (int) nuoviDeceduti;
27
28            // Verifica per evitare che i valori diventino negativi
29            sani = Math.max(sani, 0);
30            infetti = Math.max(infetti, 0);
31
32            // Correzione per assicurarsi che la somma delle variabili non superi
    la popolazione totale
33            if (sani + infetti + guariti + deceduti > popolazioneTotale) {
34                int eccedenza = (sani + infetti + guariti + deceduti) -
    popolazioneTotale;
35                infetti -= eccedenza; // Sottrae l'eccedenza dagli infetti
36            }
37
38            // Stampa lo stato aggiornato al giorno corrente
39            cornice();
40            System.out.println("Giorno " + i + ": Sani = " + sani + ", Infetti = "
    + infetti + ", Guariti = " + guariti + ", Deceduti = " + deceduti);
41            System.out.println("La somma di tutte le variabili e': " + (sani +
    infetti + guariti + deceduti) + "\n");
42            cornice();
43
44            Thread.sleep(2000); // Pausa tra un giorno e l'altro per simulare il
    passare del tempo
```



```

45     }
46 }

```

Codice 2.6: simulazioneMarkov.

Ad ogni giorno della simulazione, il codice calcola il numero di individui che transizionano tra gli stati in base alle probabilità di transizione. Ecco come viene gestito il processo:

- *Nuovi Infetti*: Determinati come una frazione della popolazione sana in base a *probTransizioneInfetto* (20%). Ad esempio, con 1000 individui sani, 200 diventeranno infetti,
- *Nuovi Guariti*: Determinati come una frazione degli infetti, secondo *probTransizioneGuarito* (80%). Con 200 infetti, guariranno 160 individui,
- *Nuovi Deceduti*: Calcolati come una frazione degli infetti in base a *probTransizioneDeceduto* (3%). Ad esempio, con 200 infetti, 6 individui decederanno,
- *Reinfezione*: Alcuni individui guariti possono essere nuovamente infettati secondo *probTransizioneReInfezione* (15%). Ad esempio con 200 guariti, 40 individui diventeranno infetti.

La popolazione viene aggiornata sottraendo i nuovi infetti dai sani, aggiungendo i nuovi infetti, guariti e deceduti agli stati appropriati.

2.2.2 Risultati

Di seguito, un esempio di output del codice:

```

1
2 Premere invio per iniziare la Simulazione!
3 +-----+
4 In questa simulazione, tentero' di fare una previsione, basata sui meri dati
   numerici, dell'andamento ipotetico del 'Covid-19'
5 Si prende di riferimento un periodo di analisi dal 24/02/2020 al 01/12/2023
6 In particolare, quello che servira' sono le probabilita' di transizione da uno
   stato ad un altro.
7 Quelle da noi prese in considerazione saranno:
8
9 Probabilita' di transizione Guarito: Useremo un valore fissato a: 80.0%
10
11 Probabilita' di transizione Deceduto: Useremo un valore fissato a: 3.0%
12
13 Probabilita' di transizione Infetto: Useremo un valore fissato a: 20.0%
14
15 Probabilita' di re-infezione: Useremo un valore fissato a: 15.0%

```

```

16
17 Premi invio per continuare...
18 +-----+
19 Ora mi serve sapere la dimensione della popolazione totale.
20 Inserisci la dimensione della popolazione totale: 10000
21 +-----+
22 Vorrei che decidessi quanti giorni durera' la simulazione, fino ad un massimo di un
    mese [31 giorni]!
23 Inserisci il numero di giorni: 4
24 +-----+
25 Una volta ottenute queste probabilita', avviamo la nostra simulazione.
26 Prendiamo di riferimento una popolazione di 10000 abitanti sani e scegliamo in
    maniera randomica
27 che una parte della popolazione si ammali.
28 Scelto un lasso di tempo di 4 giorni, vediamo come procede la nostra generazione!
29 +-----+
30 Il Primo giorno, la popolazione e' totalmente sana.
31 Giorno 0: Sani = 10000, Infetti = 0, Guariti = 0, Deceduti = 0
32 +-----+
33 Giorno 1: Sani = 8000, Infetti = 2000, Guariti = 0, Deceduti = 0
34 La somma di tutte le variabili e': 10000
35 +-----+
36 Giorno 2: Sani = 6400, Infetti = 1940, Guariti = 1600, Deceduti = 60
37 La somma di tutte le variabili e': 10000
38 +-----+
39 Giorno 3: Sani = 5120, Infetti = 1610, Guariti = 3152, Deceduti = 118
40 La somma di tutte le variabili e': 10000
41 +-----+
42 Giorno 4: Sani = 4096, Infetti = 1298, Guariti = 4440, Deceduti = 166
43 La somma di tutte le variabili e': 10000
44 +-----+
45 La simulazione e' terminata, procedi con premere invio per terminare

```

Codice 2.7: Output Markov su dati numerici.

2.2.3 Diagrammi UML

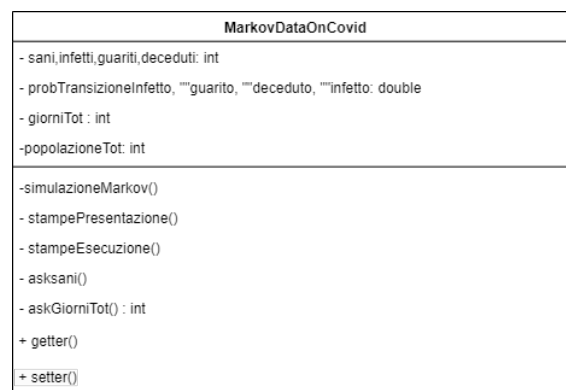


Figura 2.2: Class Diagram.

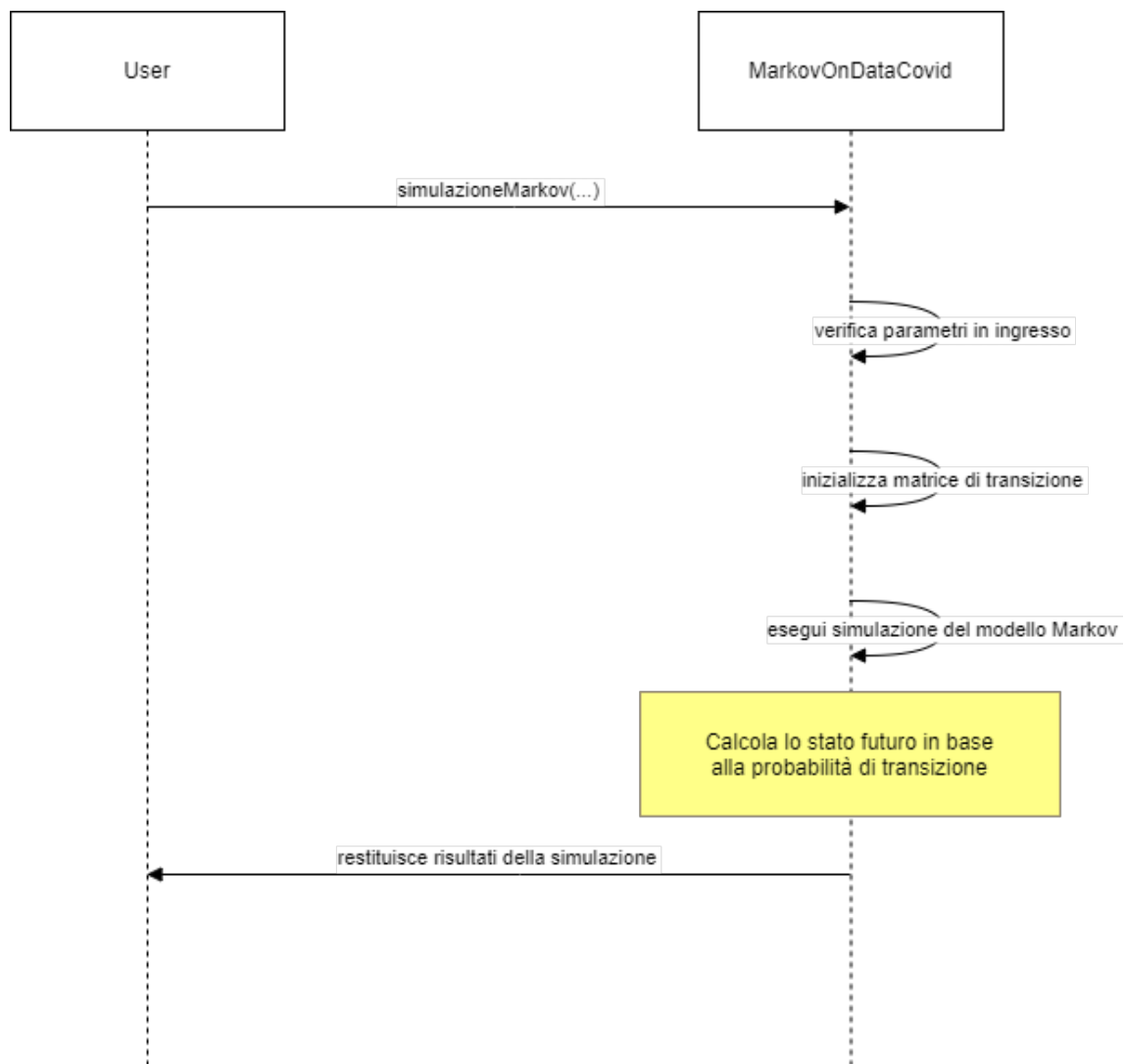


Figura 2.3: Sequence Diagram.

2.3 Terzo caso d'uso: Paradigma Client-Server

2.3.1 Implementazione

Panoramica generale

In questa implementazione, applicheremo il concetto di catena di Markov al paradigma client-server. Simuleremo una richiesta di risorse, fatta da vari client, ad un singolo server, il quale risponderà e manderà le informazioni richieste al client. Per ogni **STEP**, definiti in modo *random*, vi saranno *random* client che richiedono risorse. Questi sono definiti in un certo range di numeri interi. Inoltre, per ogni **STEP**, gli stati verranno rappresentati dal numero di client in coda per richiedere informazioni. Si procede nella creazione di una tabella di transizione, la quale crea una probabilità per ogni stato. Si sceglierà quindi di usare lo stato con la probabilità più alta, corrispondente al numero di client che richiederà risorse nella successiva iterazione. Anche in questo caso, le transizioni da uno stato all'altro del numero di clienti non dipende dalla sua storia di transizioni passate ma solo dal suo stato attuale, ovvero il principio di Markov. Una volta finita la simulazione, si procederà a realizzare un *istogramma*, per valutare tempi di risposta per ogni cliente e tempi di risposta totali per ogni client. Tutte queste informazioni saranno salvate in un **QRCode** e inviate su *Google Drive*. Questa casistica è volta ad esplorare una simulazione della realtà, nella quale diversi persone, i client, si mettono in coda per richiedere o modificare risorse ad un server. I client verranno identificati come *clienti* dal server.

Definizione probabilità

Per capire al meglio come siano gestite le probabilità, introduciamo, come precedentemente citato, la *matrice di transizione*. Tale matrice è definita in un metodo, ovvero **'generateTransitionMatrix(int currentState)'**: Tale metodo crea una matrice di transizione per ogni *STEP*. Per ogni stato, verrà generata una probabilità in modo **random**. Ogni riga è normalizzata per garantire che la somma delle probabilità sia uguale a 1, rispettando le proprietà delle probabilità. Di seguito, un esempio di come verrà visualizzata una matrice:

	0	1	2	3	4
0	[0,3308]	[0,3543]	[0,2049]	[0,0076]	[0,1024]
1	[0,0516]	[0,3330]	[0,3594]	[0,1059]	[0,1501]
2	[0,3445]	[0,1140]	[0,0823]	[0,0744]	[0,3849]

```

3 | [ 0,0539] [ 0,2636] [ 0,1545] [ 0,4002] [ 0,1278]
4 | [ 0,1389] [ 0,3419] [ 0,1616] [ 0,0142] [ 0,3434]
- - - - -

```

La matrice viene interpretata nel seguente modo: ogni riga rappresenta le probabilità di transizione dallo stato corrispondente verso gli altri stati. Ad esempio, se ci troviamo nello stato 3 (*come definito in input*), possiamo leggere le probabilità associate ai passaggi verso ciascuno stato:

```

3 | [ 0,0539] [ 0,2636] [ 0,1545] [ 0,4002] [ 0,1278]

```

- 0.0539 è la probabilità di passare dallo stato 3 allo stato 0, ossia nessun client in coda.
- 0.2636 è la probabilità di passare dallo stato 3 allo stato 1, quindi con un client in coda.
- 0.1545 è la probabilità di passare dallo stato 3 allo stato 2, ovvero con due client in coda.
- 0.4002 è la probabilità di rimanere nello stato 3, ossia con tre client in coda.
- 0.1278 è la probabilità di passare dallo stato 3 allo stato 4, quindi con quattro client in coda.

Si osserva facilmente che la probabilità più alta è 0.4002, perciò nel prossimo ciclo di **STEP** il numero di client che genererà richieste sarà proprio lo stato 3, ossia tre client effettueranno richieste al server.

Classi di supporto

Per facilitare l'esecuzione del codice, sono state create tre classi molto importanti:

- **ClientData**, contenete *ID*, *waitTime* e *serviceTime*,
- **PersonalizedClient**, il modello personalizzato di Client, ideato per rispondere in maniera consona al server. Il client presenta dei metodi **start()** e **stop()**, omonimi della loro funzione. A seguire, il metodo più importante della classe: **public void sendRequest(...)** Questo metodo viene definito come:

```

1      public void sendRequest(String targetUrl, String method, List<
ClientData> clientDataList, int indexOfClient) throws InterruptedException
, IOException {
2          System.out.println("\n");
3          System.out.println(method + " " + targetUrl + " HTTP/1.1");
4          Thread.sleep(1000);
5          setTargetUrl(targetUrl);
6
7          // Configura e stampa gli header
8          configureHeaders();
9
10         // Seleziona un JSON casuale
11         String jsonData = getRandomJsonData();
12
13         // Se ci sono dati JSON (per POST o PUT), li stampiamo
14         if (jsonData != null) {
15             System.out.println("\n");
16             System.out.println("Request Body:");
17             System.out.println(jsonData);
18             System.out.println();
19         }
20         Thread.sleep(1000);
21
22         // Ottieni il cliente specificato dall'indice della clientDataList
23         ClientData clientData = clientDataList.get(indexOfClient - 1);
24
25         // Simulazione tempi di attesa e servizio casuali
26         long actualWaitTime = simulateWaitTime();
27         long actualServiceTime = simulateServiceTime();
28
29         // Imposta i tempi nel ClientData
30         clientData.setWaitTime(actualWaitTime);
31         clientData.setServiceTime(actualServiceTime);
32
33         // Aggiorna clientDataList con il nuovo ClientData modificato
34         clientDataList.set(indexOfClient - 1, clientData);
35
36         // Stampa un codice di risposta verosimile
37         int responseCode = simulateResponseCode(method);
38         System.out.println("Risposta da parte del server...\n");
39         Thread.sleep(1000);
40         System.out.println("HTTP/1.1 " + responseCode + " " +
getResponseMessage(responseCode));
41         Thread.sleep(1000);
42         // Stampa i tempi in maniera realistica
43         System.out.println("X-Wait-Time: " + actualWaitTime + " ms");
44         Thread.sleep(1000);
45         System.out.println("X-Service-Time: " + actualServiceTime + " ms");
46         System.out.println("
+-----+");
47         Thread.sleep(1000);
48         System.out.println("Richiesta mandata al server...attesa elaborazione"
);
49         Thread.sleep(2000);
50         PersonalizedServer.handleClient(method, targetUrl);
51     }

```

52

Codice 2.8: sendRequest.

Questo metodo gestisce le richieste dei client, con relative stampe funzionali.

- **PersonalizedServer**, il modello personalizzato di Server, ideato con una struttura *HTTP*, per gestire, lavorare e rispondere in maniera congrua alle richieste dei client. Il client presenta dei metodi **start()** e **stop()**, omonimi della loro funzione. A seguire, il metodo **handleclient(...)** e **generateHttpResponse(...)**

```
1      public static void handleClient(String method, String targetUrl)
2      throws InterruptedException {
3          // Crea una richiesta dal client
4          System.out.println("Sono il server..Sto ricevendo qualcosa...\n");
5          Thread.sleep(1000);
6
7          String request = method + " " + targetUrl + " HTTP/1.1";
8          System.out.println("Ricevuto: " + request);
9          Thread.sleep(2000);
10
11         // Determina il tipo di richiesta e genera una risposta appropriata
12         String response;
13         if (method.equals("GET")) {
14             response = generateHttpResponse(200, "OK", "Ecco il contenuto
15             richiesto.");
16         } else if (method.equals("POST")) {
17             response = generateHttpResponse(201, "Created", "Risorsa creata
18             con successo.");
19         } else if (method.equals("DELETE")) {
20             response = generateHttpResponse(204, "No Content", "Risorsa
21             eliminata.");
22         } else {
23             response = generateHttpResponse(400, "Bad Request", "La richiesta
24             non e' valida.");
25         }
26         // Invia la risposta al client
27         System.out.println("Risposta: " + response);
28     }
29
30     // Genera una risposta HTTP simulata
31     private static String generateHttpResponse(int statusCode, String
32     statusMessage, String bodyContent) throws InterruptedException {
33         // Usa ZonedDateTime per includere l'offset
34         ZonedDateTime zdt = ZonedDateTime.now(ZoneOffset.UTC); // Specifica l'
35         offset desiderato
36         DateTimeFormatter dtf = DateTimeFormatter.ofPattern("EEE, dd MMM yyyy
37         HH:mm:ss z");
38
39         String dateHeader = dtf.format(zdt);
```

```

32
33     StringBuilder response = new StringBuilder();
34     response.append("HTTP/1.1 ").append(statusCode).append(" ").append(
statusMessage).append("\r\n");
35     response.append("Date: ").append(dateHeader).append("\r\n");
36     response.append("Content-Type: eseguibili.text/html; charset=UTF-8\r\n
");
37     response.append("Content-Length: ").append(bodyContent.length()).
append("\r\n");
38     response.append("Connection: close\r\n");
39     response.append("\r\n");
40     response.append(bodyContent);
41
42     return response.toString();
43 }
44

```

Codice 2.9: PersonalizedServer.

Questi metodi creano delle risposte sulla base delle richieste dei client.

Main

Il metodo principale che si occupa di tutto è proprio il main. Si occupa di far partire client e server, di generare le matrici, di gestire la logica per ogni **STEP** e di creare gli *istogrammi*, i **Qrcode** e l'*upload*.

```

1     public static void main(String[] args) throws Exception {
2         System.out.println("Premi invio per cominciare la simulazione.");
3         attendiInput(); // Attende input dell'utente
4         stampePresentazione(); // Mostra la presentazione della simulazione
5         attendiInput(); // Attende input dell'utente per continuare
6
7         // Inizio della simulazione
8         System.out.println("
*+-----+*");
9         System.out.println("INIZIAMO LA SIMULAZIONE DEL PROCESSO");
10        Thread.sleep(1000);
11        System.out.println("\nCome dati in ingresso, scegliamo "+STEPS+" STEPS\n");
12        Thread.sleep(1000);
13        System.out.println("Per la prima interazione, scegliamo: "+MAX_CLIENTS+ "
clienti\n");
14        setCurrentState(MAX_CLIENTS);
15        Thread.sleep(1000);
16
17        client.start(); // Avvio del client simulato
18        Thread.sleep(1000);
19
20        server.start(); // Avvio del server simulato
21        Thread.sleep(1000);
22

```



```

23     System.out.println("Connessione stabilita con il server all'indirizzo " + "
    localhost" + ":" + "8080" + ".\n");
24     System.out.println("Client ora connessi al server.\n");
25     Thread.sleep(1000);
26
27     // Inizializzazione della matrice di transizione
28     setTransitionMatrix(generateTransitionMatrix());
29     stateOfMatrix = new StateofMatrix(currentState, transitionMatrix); // Passa
    la matrice al costruttore
30
31     for (int step = 1; step <= STEPS; step++) {
32         if(step==1){
33             cornice();
34             System.out.println("INIZIO RICHIESTE");
35             cornice();
36             System.out.println("\n");
37         }
38
39         // Simula la coda con i clienti e il server
40         simulateQueueWithClientsAndServer();
41
42         //stampa la matrice generata per questo step, tranne per l'ultimo!
43         if(step != STEPS) {
44             System.out.println("\n");
45             System.out.println("La matrice che verra' usata per il prossimo
    step e' :");
46             printTransitionMatrix(getTransitionMatrix());
47         }
48         // Aggiorna lo stato per determinare il numero di clienti nel prossimo
    passo
49         MAX_CLIENTS = stateOfMatrix.updatingState(getCurrentState());
50         setCurrentState(MAX_CLIENTS);
51
52         if(step != STEPS) {
53             System.out.println("Il server ha decretato che per la fase
    successiva ci saranno " + MAX_CLIENTS + " clienti!\n");
54         }
55         //genera la matrice generata per questo step, tranne per l'ultimo!
56         if(step != STEPS) {
57             generateTransitionMatrix();
58         }
59
60         System.out.println("\n");
61     }
62     // Stampa la lista finale dei clienti dopo la simulazione
63     System.out.println("L'ultimo cliente e' stato servito.\n");
64     cornice();
65     System.out.println("\n");
66     System.out.println("Recupero delle informazioni di tutti i clienti serviti!
    ");
67     printClientDataList();
68     cornice();
69     System.out.println("\n");
70
71     OutputRedirector.closeOutputFile(); // Chiudi il file di output
72     client.stop();
73     server.stop();

```

```

74
75     cornice();
76     System.out.println("\n");
77     // Analisi dei dati dei clienti
78     ServiceAndAverageTime serviceAndAverageTime = new ServiceAndAverageTime();
79     serviceAndAverageTime.analyzeData(clientDataList);
80
81     cornice();
82     System.out.println("\n");
83
84     System.out.println("PROCEDEREMO ORA A COMPRIMERE I DATI IN UNA ZIP, E A
CARICARLI SU GOOGLE DRIVE");
85     // Chiamate per file QR, compressione e upload
86     FileDivider.main();
87     WinRARCompression.main();
88     GoogleDriveUploader.main();
89     QRCodeGenerator.main();
90
91     System.out.println("Chiudi le finestre create , poi premi invio per
terminare la simulazione...");
92     attendiInput();
93 }
94

```

Codice 2.10: Main.

Particolare attenzione va portata al metodo `simulateQueueWithClientsAndServer()`

```

1     public static void simulateQueueWithClientsAndServer() throws IOException,
InterruptedException {
2         String[] methods = {"GET", "POST", "PUT", "DELETE"}; // Metodi HTTP
simulati
3         Random random = new Random(); // Generatore di numeri casuali
4
5         // Ciclo per simulare MAX_CLIENTS clienti
6         for (int i = 0; i < MAX_CLIENTS; i++) {
7             String randomMethod = methods[random.nextInt(methods.length)]; //
Metodo HTTP casuale
8             String targetUrl = "http://localhost:8080/" + chooseRandomPath(); //
URL casuale
9
10            // Crea un oggetto ClientData e aggiungilo alla lista
11            ClientData clientData = new ClientData(clientIndex, 100, 100); //
Assegna l'indice globale al cliente
12            clientDataList.add(clientData);
13
14            // Invia la richiesta al server
15
16            cornice();
17            System.out.println("Invio della richiesta al server del cliente numero:
"+clientIndex+"...");
18            cornice();

```

```

19         Thread.sleep(2000);
20         client.sendRequest(targetUrl, randomMethod, clientDataList, clientIndex
21     );
22
23     // Recupera e stampa i tempi di attesa e di servizio
24     long waitTime = clientData.getWaitTime();
25     long serviceTime = clientData.getServiceTime();
26
27     System.out.println("Recupero delle informazioni del server del cliente:
28     "+ clientData.getClientNumber());
29
30     stampaInfoPerCliente(waitTime, serviceTime, clientData.getClientNumber
31     ());
32
33     clientIndex++; // Incrementa l'indice per il prossimo cliente
34 }
35 }

```

Codice 2.11: simulateQueueWithClientsAndServer.

Questo metodo gestisce i client, le richieste e aggiorna la lista di client per tenere traccia dei tempi per ogni cliente.

2.3.2 Risultati

Di seguito, un esempio di output del codice:

```

1 Premi invio per cominciare la simulazione.
2
3 In questa simulazione,simuleremo una catena di markov su una coda di clienti.
4 I clienti effettuano richieste al server e ne verranno misurati i tempi di attesa e
5   di servizio.
6
7 Definiamo una variabile STEP, che rappresenta quante volte le code di clienti
8   verranno create.
9 Per ogni step, ci saranno Random clienti , definiti da MAX_CLIENT, con max 4
10  clienti per STEP.
11
12 MAX_CLIENT sara' gestito con la creazione di una matrice che terra' conto delle
13  proprieta' delle catene di Markov.
14 Questa matrice verra' stampata, rappresentando per righe il numero di clienti e per
15  colonne la probabilita' di passaggio di stato ad N clienti.
16
17 Premi invio per procedere con la simulazione.
18 +-----+
19 INIZIAMO LA SIMULAZIONE DEL PROCESSO
20
21 Come dati in ingresso, scegliamo 2 STEPS
22
23 Per la prima interazione, scegliamo: 3 clienti
24

```

```
20 I client sono stati avviati. Attendono la connessione al server...
21
22 Il server e' stato avviato.
23
24 Inizio dell'ascolto delle richieste dai client...
25
26 Il server e' ora in ascolto sulla porta 8080.
27
28 Connessione stabilita con il server all'indirizzo localhost:8080.
29
30 Client ora connessi al server.
31 +-----+
32 INIZIO RICHIESTE
33 +-----+
34 Invio della richiesta al server del cliente numero: 1...
35 +-----+
36 DELETE http://localhost:8080//faq/help HTTP/1.1
37 Content-Type: application/pdf
38 Authorization: Bearer Token: 684882672742209
39 Accept-Encoding: br
40 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:89.0) Gecko/20100101 Firefox/89.0
41 Cache-Control: max-age=0
42 Accept: eseguibili.text/html
43
44 Request Body:
45 {"messageId": 404, "sender": "johndoe@example.com", "receiver": "janedoe@example.
    com", "content": "Ciao, come stai?"}
46
47 Risposta da parte del server...
48
49 HTTP/1.1 404 Not Found
50 X-Wait-Time: 801 ms
51 X-Service-Time: 1314 ms
52 +-----+
53 Richiesta mandata al server...attesa elaborazione
54 Sono il server..Sto ricevendo qualcosa...
55
56 Ricevuto: DELETE http://localhost:8080//faq/help HTTP/1.1
57 Risposta: HTTP/1.1 204 No Content
58 Date: ven, 25 ott 2024 14:31:38 Z
59 Content-Type: eseguibili.text/html; charset=UTF-8
60 Content-Length: 18
61 Connection: close
62
63 Risorsa eliminata.
64 Recupero delle informazioni del server del cliente: 1
65
66 Cliente 1: Tempo di attesa: 801 ms, Tempo di servizio: 1314 ms
67 +-----+
68 Invio della richiesta al server del cliente numero: 2...
69 +-----+
70 PUT http://localhost:8080//faq/help HTTP/1.1
71 Content-Type: image/png
72 Authorization: Basic Token: 508318448687523
73 Accept-Encoding: br
74 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
    like Gecko) Chrome/87.0.4280.66 Safari/537.36
```

```
75 Cache-Control: must-revalidate
76 Accept: application/xml
77
78 Request Body:
79 {"reviewId": 303, "productId": 101, "rating": 5, "comment": "Ottimo prodotto!"}
80
81 Risposta da parte del server...
82
83 HTTP/1.1 201 Created
84 X-Wait-Time: 876 ms
85 X-Service-Time: 1461 ms
86 +-----+
87 Richiesta mandata al server...attesa elaborazione
88 Sono il server..Sto ricevendo qualcosa...
89
90 Ricevuto: PUT http://localhost:8080//faq/help HTTP/1.1
91 Risposta: HTTP/1.1 400 Bad Request
92 Date: ven, 25 ott 2024 14:31:54 Z
93 Content-Type: eseguibili.text/html; charset=UTF-8
94 Content-Length: 26
95 Connection: close
96
97 La richiesta non e' valida.
98 Recupero delle informazioni del server del cliente: 2
99
100 Cliente 2: Tempo di attesa: 876 ms, Tempo di servizio: 1461 ms
101 +-----+
102 Invio della richiesta al server del cliente numero: 3...
103 +-----+
104 PUT http://localhost:8080//http2/http1 HTTP/1.1
105 Content-Type: application/zip
106 Authorization: Basic Token: 707691969852732
107 Accept-Encoding: identity
108 User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 14_0 like Mac OS X) AppleWebKit
    /605.1.15 (KHTML, like Gecko) Version/14.0 Mobile/15E148 Safari/604.1
109 Cache-Control: no-cache
110 Accept: application/json
111
112 Request Body:
113 {"profileId": 606, "userId": 1, "bio": "Appassionato di tecnologia e programmazione
    ."}
114
115 Risposta da parte del server...
116
117 HTTP/1.1 201 Created
118 X-Wait-Time: 717 ms
119 X-Service-Time: 1316 ms
120 +-----+
121 Richiesta mandata al server...attesa elaborazione
122 Sono il server..Sto ricevendo qualcosa...
123
124 Ricevuto: PUT http://localhost:8080//http2/http1 HTTP/1.1
125 Risposta: HTTP/1.1 400 Bad Request
126 Date: ven, 25 ott 2024 14:32:11 Z
127 Content-Type: eseguibili.text/html; charset=UTF-8
128 Content-Length: 26
129 Connection: close
```

```

130
131 La richiesta non e' valida.
132 Recupero delle informazioni del server del cliente: 3
133
134 Cliente 3: Tempo di attesa: 717 ms, Tempo di servizio: 1316 ms
135
136 La matrice che verra' usata per il prossimo step e' :
137 +-----+
138         0         1         2         3         4
139 - - - - -
140 0 | [ 0,3594] [ 0,1266] [ 0,0785] [ 0,0857] [ 0,3498]
141 1 | [ 0,2555] [ 0,2337] [ 0,1208] [ 0,1270] [ 0,2630]
142 2 | [ 0,2070] [ 0,0101] [ 0,1332] [ 0,1737] [ 0,4760]
143 3 | [ 0,2926] [ 0,4675] [ 0,1895] [ 0,0253] [ 0,0250]
144 4 | [ 0,0004] [ 0,2314] [ 0,3264] [ 0,0780] [ 0,3639]
145 - - - - -
146
147 +-----+
148 Il server ha decretato che per la fase successiva ci saranno 1 clienti!
149 +-----+
150
151 Invio della richiesta al server del cliente numero: 4...
152 +-----+
153 GET http://localhost:8080//products HTTP/1.1
154 Content-Type: application/zip
155 Authorization: Bearer Token: 845594994509617
156 Accept-Encoding: gzip, deflate
157 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (
     KHTML, like Gecko) Chrome/91.0.4472.114 Safari/537.36
158 Cache-Control: no-cache
159 Accept: application/json
160
161 Request Body:
162 {"eventId": 505, "eventName": "Conferenza", "date": "2024-10-01T09:00:00Z", "
      location": "Sala conferenze A"}
163
164 Risposta da parte del server...
165
166 HTTP/1.1 200 OK
167 X-Wait-Time: 851 ms
168 X-Service-Time: 1928 ms
169 +-----+
170 Richiesta mandata al server...attesa elaborazione
171 Sono il server..Sto ricevendo qualcosa...
172
173 Ricevuto: GET http://localhost:8080//products HTTP/1.1
174 Risposta: HTTP/1.1 200 OK
175 Date: ven, 25 ott 2024 14:32:37 Z
176 Content-Type: eseguibili.text/html; charset=UTF-8
177 Content-Length: 28
178 Connection: close
179
180 Ecco il contenuto richiesto.
181 Recupero delle informazioni del server del cliente: 4
182
183 Cliente 4: Tempo di attesa: 851 ms, Tempo di servizio: 1928 ms
184

```

```

185 L'ultimo cliente e' stato servito.
186 +-----+
187 Recupero delle informazioni di tutti i clienti serviti!
188 Dati dei clienti nella lista:
189
190 Cliente 1: Tempo di attesa: 801 ms, Tempo di servizio: 1314 ms
191
192 Cliente 2: Tempo di attesa: 876 ms, Tempo di servizio: 1461 ms
193
194 Cliente 3: Tempo di attesa: 717 ms, Tempo di servizio: 1316 ms
195
196 Cliente 4: Tempo di attesa: 851 ms, Tempo di servizio: 1928 ms
197 +-----+
198 I client stanno chiudendo la connessione al server...
199
200 I client sono stati fermati con successo.
201
202 Il server sta chiudendo tutte le connessioni attive...
203
204 Il server e' stato fermato con successo.
205
206 Tutte le connessioni sono state chiuse. Il server non e' piu' in ascolto.
207 +-----+
208 Analizziamo ora il tempo di attesa medio e il tempo di servizio medio!
209
210 Tempo di attesa medio: 811.25 ms
211 Tempo di servizio medio: 1504.75 ms
212 +-----+
213 PROCEDEREMO ORA A COMPRIMERE I DATI IN UNA ZIP, E A CARICARLI SU GOOGLE DRIVE
214 I TUOI FILE SONO PRONTI A DIVENTARE QR
215 Compressione completata con codice di uscita: 0
216 Se 0, ha avuto successo!
217 Upload completato con successo!
218 QR Code immagine salvato con successo!
219
220 Chiudi le finestre create , poi premi invio per terminare la simulazione...

```

Codice 2.12: Output Markov con Paradigma Client-Server.

Di seguito gli istogrammi generati per Wait-Time e Service-Time.

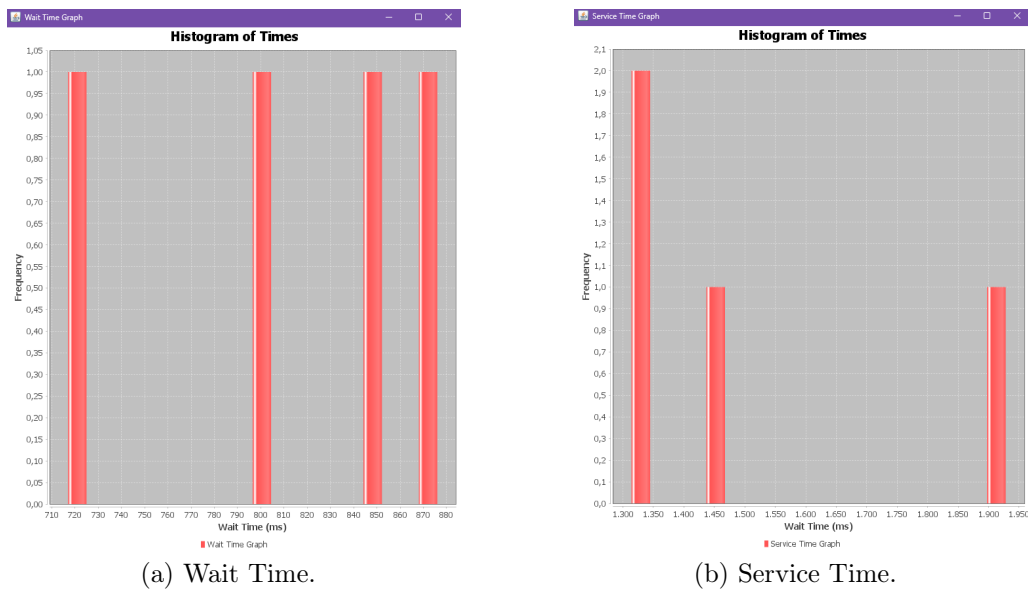


Figura 2.4: Istogrammi delle prestazioni del server.

2.3.3 Diagrammi UML

Questa sezione descrive la struttura del sistema dal punto di vista del linguaggio UML. Troviamo quattro classi principali:

- 1. MarkovQueueSimulation,
- 2. PersonalizedClient,
- 3. PersonalizedServer,
- 4. ClientData.

MarkovQueueSimulation

La classe MarkovQueueSimulation gestisce l'intero processo di simulazione, inclusa la gestione dei client, del server e della transizione tra stati.

Attributo	Descrizione
maxClients	Numero massimo di client nella simulazione
STEPS	Numero di iterazioni massime
currentState	Conserva lo stato corrente
transitionMatrix	Matrice delle probabilità di transizione tra stati
clientDataList	Lista contenente i dati di ogni client
clientIndex	Indice globale per i client
stampe varie	Stampe di vario tipo e genere

Tabella 2.1: Attributi della classe MarkovQueueSimulation.

Metodo	Descrizione
simulateQueueWithClientsAndServer()	Gestione richieste
generateTransitionMatrix()	Genera la matrice
updatingState()	Gestisce transizione tra stati
printTransitionMatrix()	Stampa della matrice
printClientDataList()	Stampa i dati dei client
updateMatrix()	Aggiorna la matrice di transizione

Tabella 2.2: Metodi della classe `MarkovQueueSimulation`.

PersonalizedClient

La classe `PersonalizedClient` rappresenta ogni client nella simulazione, gestendo i tipi di richieste HTTP e i tempi di attesa e servizio.

Attributo	Descrizione
serverAddress	Indirizzo localhost
serverPort	Porta
isRunning	Per conoscere se il client è disponibile

Tabella 2.3: Attributi della classe `PersonalizedClient`.

Metodo	Descrizione
sendRequest(...)	Invia richieste al server simulando attese e servizio
handleClient(...)	Gestisce risposta del client
generateHttpResponse(...)	Genera la risposta del client
configureHeaders()	Gestione degli headers HTTP
getRandomJsonData()	Gestione dei dati Json
start() e stop()	Per avviare e fermare il client

Tabella 2.4: Metodi della classe `PersonalizedClient`.

PersonalizedServer

Metodo	Descrizione
handleClient(...)	Gestisce richieste dai client
generateHttpResponse(...)	Gestisce la generazione delle risposte
start() e stop()	Per avviare e fermare il Server

Tabella 2.5: Metodi della classe `PersonalizedServer`.

Attributo	Descrizione
serverAddress	Indirizzo localhost
serverPort	Porta
isRunning	Per conoscere se il server è disponibile

Tabella 2.6: Attributi della classe `PersonalizedServer`.

ClientData

La classe `ClientData` contiene i dati relativi al tempo di attesa, risposta e servizio per ciascuna richiesta.

Attributo	Descrizione
clientNumber	ID client
waitTime	Tempo di risposta di attesa dal client
serviceTime	Tempo di servizio offerto dal server

Tabella 2.7: Attributi della classe `ClientData`.

Relazioni tra le Classi

- La classe `MarkovQueueSimulation` ha una relazione uno-a-molti con `PersonalizedClient` e `ClientData`,
- Ha una relazione uno-a-uno con `PersonalizedServer`.

Capitolo 3

Considerazioni Finali

Le catene di Markov non solo offrono un framework matematico per comprendere e analizzare fenomeni complessi, ma si rivelano anche strumenti versatili e pratici in una gamma di contesti. Di seguito alcuni esempi concreti che illustrano la loro applicazione in ambiti specifici nella realtà:

Analisi Testuale (NLP)

Nel campo dell'elaborazione del linguaggio naturale, le catene di Markov sono fondamentali per tecniche come il tagging delle parti del discorso e il riconoscimento vocale. Ad esempio, i modelli Hidden Markov sono ampiamente utilizzati per determinare la probabilità che una parola appartenga a una determinata categoria grammaticale, considerando le parole circostanti. Applicazioni pratiche di questo approccio possono essere trovate in strumenti di traduzione automatica e assistenti vocali, come Google Translate e Siri, dove le previsioni sui tag grammaticali sono essenziali per migliorare l'accuratezza e la fluidità del linguaggio generato [5].

Analisi di Dati

Le catene di Markov sono anche utilizzate nell'analisi di serie temporali, come nel contesto della previsione dei mercati finanziari. Attraverso l'analisi delle transizioni tra stati di prezzo, gli investitori possono stimare la probabilità di cambiamenti nel valore delle azioni, aiutando a prendere decisioni informate sulle operazioni di acquisto e vendita. Ad esempio, molte strategie di trading algoritmico si basano su modelli di Markov per analizzare i movimenti dei prezzi e identificare opportunità di investimento [6].

Paradigma Client-Server

Nei sistemi client-server, le catene di Markov possono ottimizzare la gestione delle richieste dei client. Utilizzando queste catene, un server può prevedere il carico di lavoro futuro e adattarsi di conseguenza, distribuendo le richieste su più server e prevenendo sovraccarichi. Il Cloud Computing rappresenta un esempio significativo in questo contesto, in quanto offre un paradigma computazionale che semplifica la gestione della tecnologia dell'informazione. Secondo Kephart e Chess (2011), il Cloud Computing, basato su tecnologie di virtualizzazione, facilita servizi facilmente configurabili e scalabili che possono essere acquistati secondo necessità. Ciò ha un impatto diretto sull'efficienza operativa delle architetture server [7].

Le catene di Markov dimostrano di essere strumenti straordinari, non solo per la loro capacità di rappresentare sistemi dinamici, ma anche per la loro applicabilità pratica in vari settori. Questo progetto ha messo in luce la loro versatilità, dimostrando come possano essere utilizzate per analizzare, prevedere e ottimizzare processi in contesti reali, rendendo evidente il loro valore nelle scienze applicate e nell'ingegneria.

Capitolo 4

Ringraziamenti

Ringrazio tutti coloro che hanno reso possibile il raggiungimento di questo traguardo tanto atteso.

- **Mamma e papà:** Grazie per la pazienza infinita e il continuo supporto; spero di ripagarvi mille volte tanto.
- **Mia sorella:** Che mi ha sempre fatto sentire compreso e a casa.
- **I miei amici:**
 - **Paolo:** Di solito è sconsigliato incontrare i propri eroi. Paolo ha conosciuto il suo ormai svariati anni or sono. Il 90% degli esami lo devo a lui.
 - **Mike:** Che mi ha sempre supportato nei post esami disastrosi, nei pre-esami ansiogeni e nella mia quotidianità.
 - **Marco:** Che ha sempre supportato il mio percorso, i miei studi e mi ha sempre aiutato nel suo.
 - **Martina:** Che ha ascoltato drama e traumi giornalieri della mia avventura di studi ogni giorno, facendomi sentire meno solo.
 - **Chiara:** Che non ha smesso di torturarmi un secondo in tutte le chiamate su Discord, ma senza la quale non sarei arrivato a fine mese con la sanità mentale. Spoiler: l'ho persa comunque.
 - **Debora e Sabrina:** Mie sorelle adottive, senza le quali non sarei chi sono oggi. Non avrei abbastanza parole da spendere per entrambe, ma spero che il mio grazie più sincero sia sufficiente a ripagarvi (Ho capito, andiamo al Garden...).

- **Cristina C.** mio punto di riferimento, ti auguro di convertire la mia eresia (illuditi pure eheh...)
- **Marta:** che mi ha supportato/sopportato e condiviso con me il tempo della triennale, l'astio verso geometria e le innumerevoli volte in cui mi hai salvato.
- **A tutti i miei amici:** a coloro che non sono stati menzionati, ma che ci sono stati per me. Grazie.
- **Professoressa Maria Manfredini:** Che ha alleviato il mio primo anno e nel tempo ha dimostrato di essere una Professoressa con la P maiuscola. Ha spronato tutti, è stata sempre cordiale e non ha mai lasciato uno studente in difficoltà. Grazie.
- **Al mio relatore:** Che ha saputo insegnare come l'essere professionali non debba per forza essere sinonimo di rigoroso.

Congratulazioni.



Bibliografia

- [1] Oxford Dictionaries. Markov Chain: Definition of Markov Chain in US English by Oxford Dictionaries. https://web.archive.org/web/20171215001435/https://en.oxforddictionaries.com/definition/us/markov_chain.
Ultimo accesso: 2024-10-21.
- [2] Aletti, Giacomo. Definizione di Proprietà di Markov. [https://www.treccani.it/enciclopedia/proprieta-di-markov_\(Enciclopedia-della-Scienza-e-della-Tecnica\)/](https://www.treccani.it/enciclopedia/proprieta-di-markov_(Enciclopedia-della-Scienza-e-della-Tecnica)/). Enciclopedia della Scienza e della Tecnica, Treccani. Ultimo accesso: 2024-10-21.
- [3] Rick Durrett. *Essentials of Stochastic Processes*. Springer Verlag, New York, 2nd edition, 2012.
- [4] J. R. Norris. *Markov Chains*. Cambridge University Press, 1998.
- [5] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. The MIT Press, Cambridge, MA, 2008.
- [6] J. Hull. *Options, Futures and Other Derivatives*. Eastern economy edition. Pearson/Prentice Hall, 2009.
- [7] Jeffrey O. Kephart and David M. Chess. Cloud computing and the role of the cloud. *IBM Journal of Research and Development*, 55(6):1–2, 2011.